🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

รายชื่อสมาชิก:

673380029-0	กิตติธัช คำโท	sec2
673380041-0	ธนัช พนมเริงไชย	sec2
673380262-4	กรมภัฏ พิริยะ	sec2
673380511-9	ณัฏดนย์ สาริกา	sec2
673380513-5	นาเดีย คิดอ่าน	sec2

👥 TEAM BREAKDOWN (5–6 students)

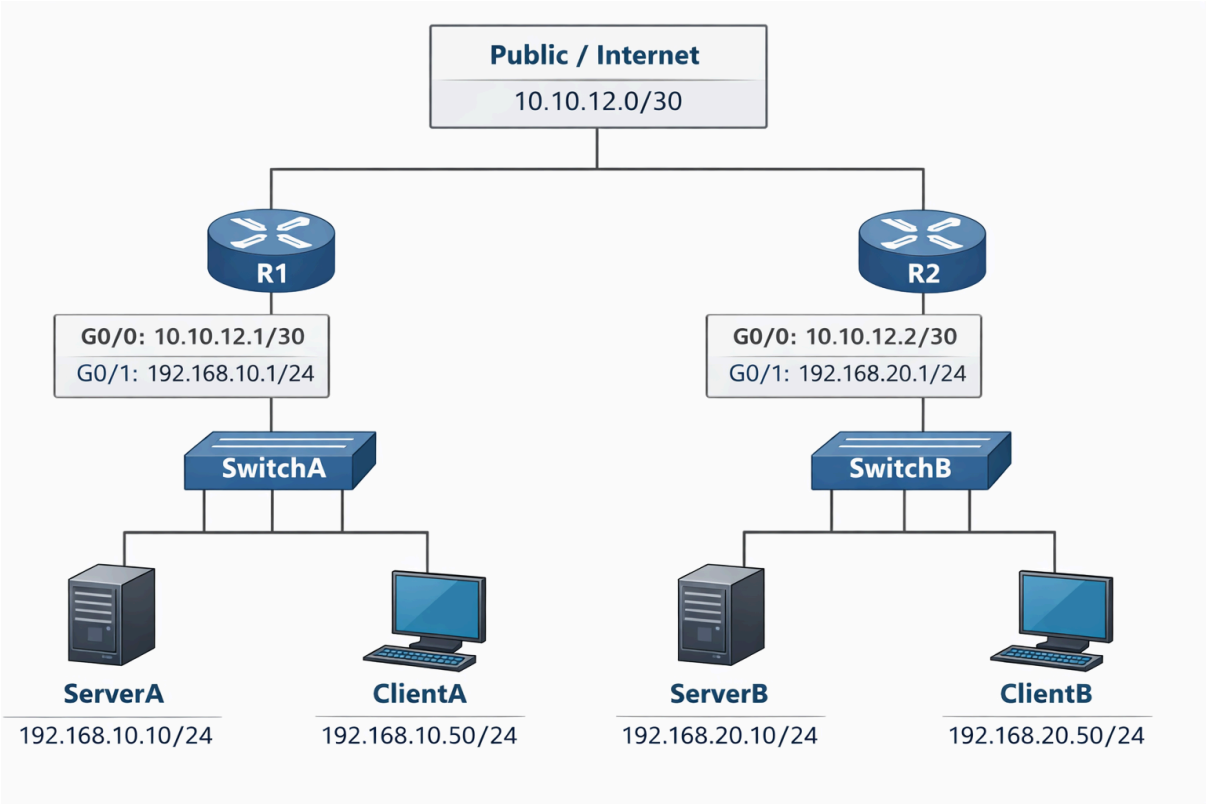
ชื่อ - รหัสนักศึกษา	Role	Responsibilities
กิตติธัช คำโท (673380029-0)	Network Architect	ออกแบบ Network Topology และตั้งค่า Router (Static Routing, NAT) ด้วย Cisco CLI
ธนัช พนมเริงไชย (673380041-0)	Network Architect	ตรวจสอบ Routing Table, NAT Translation และทดสอบการเชื่อมต่อระหว่าง LAN
กรมภัฏ พิริยะ (673380262-4)	DevOps / IaC	จัดการ Project ใน VS Code และ Deploy ระบบ mockup-infra
ณัฏดนย์ สาริกา (673380511-9)	DevOps / IaC	ตั้งค่าไฟล์ Environment (ServerA.env, ServerB.env) และจัดการ Container Deployment
นาเดีย คิดอ่าน (673380513-5)	QA / Tester	ทดสอบ API (Stateless / Stateful), ตรวจสอบ Session Behavior และบันทึกผลการทดสอบ

🗺️ UPDATED NETWORK TOPOLOGY

```python
10.10.12.0/30 “Public/Internet”
```

```
            ┌──────────────────────────┐
            |                          |
        R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
            |                          |
  Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
            |                          |
        SwitchA                      SwitchB
            |                          |
ServerA (mockup)            ServerB (mockup)
ClientA                         ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.



📍 ADDRESS TABLE

Component	Interface	IP	Mask	Role
R2	G0/0	10.10.12.1	/30	Public

Component	Interface	IP	Mask	Role
R2	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R3	G0/0	10.10.12.2	/30	Public
R3	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R2 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R2

interface g0/0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
```

```
ip nat inside source list 10 interface g0/0/0 overload

end
write memory
```

🧱 R3 CONFIGURATION (LAN B)

```
None
enable
conf t

hostname R3

interface g0/0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0/0 overload

end
write memory
```

Router แต่ละตัวทำหน้าที่เป็น Gateway ของแต่ละ LAN และเชื่อมต่อกันผ่าน Public Network เพื่อให้สามารถติดต่อกันข้ามเครือข่ายได้

ผลลัพธ์ที่ได้คืออุปกรณ์ทุกตัวสามารถเชื่อมต่อกันผ่าน Router และสามารถส่งข้อมูลระหว่าง LAN A และ LAN B ได้สำเร็จ

🧪 ROUTING & NAT VERIFICATION

On R2 & R3:

```
None
show ip route
show ip nat translations
```

```
R2#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, m - OMP
       n - NAT, Ni - NAT inside, No - NAT outside, Nd - NAT DIA
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       H - NHRP, G - NHRP registered, g - NHRP registration summary
       o - ODR, P - periodic downloaded static route, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR
       & - replicated local route overrides by connected

Gateway of last resort is not set

      192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.10.0/24 is directly connected, GigabitEthernet0/0/1
L        192.168.10.1/32 is directly connected, GigabitEthernet0/0/1
R2#
```

```
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/1 ms
R3#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, m - OMP
       n - NAT, Ni - NAT inside, No - NAT outside, Nd - NAT DIA
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       H - NHRP, G - NHRP registered, g - NHRP registration summary
       o - ODR, P - periodic downloaded static route, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR
       & - replicated local route overrides by connected

Gateway of last resort is not set

      10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C        10.10.12.0/30 is directly connected, GigabitEthernet0/0/0
L        10.10.12.2/32 is directly connected, GigabitEthernet0/0/0
S     192.168.10.0/24 [1/0] via 10.10.12.1
      192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.20.0/24 is directly connected, GigabitEthernet0/0/1
L        192.168.20.1/32 is directly connected, GigabitEthernet0/0/1
R3#show ip nat translations
Pro  Inside global         Inside local          Outside local         Outside global
icmp 10.10.12.2:5          192.168.20.1:5        10.10.12.1:5          10.10.12.1:5
Total number of translations: 1

R3#
```

From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```

การตรวจสอบ Routing Table ด้วยคำสั่ง show ip route

พบว่า Router มีเส้นทางไปยังเครือข่ายปลายทางครบถ้วน ทำให้สามารถส่ง Packet ระหว่างสอง LAN ได้

การตรวจสอบ NAT ด้วยคำสั่ง show ip nat translations

พบว่ามีการแปล IP Address จาก Private Network ไปเป็น Public Network เมื่อมีการส่งข้อมูลออกไปยังอีกฝั่งของเครือข่าย

ผลลัพธ์คือ Router สามารถทำ NAT ได้ถูกต้อง และทำให้เครื่องใน Private LAN สามารถเข้าถึงบริการจากอีกเครือข่ายได้

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠️ CLONE REPO

```shell
git clone https://github.com/boonsup/automation.git
cd automation
```

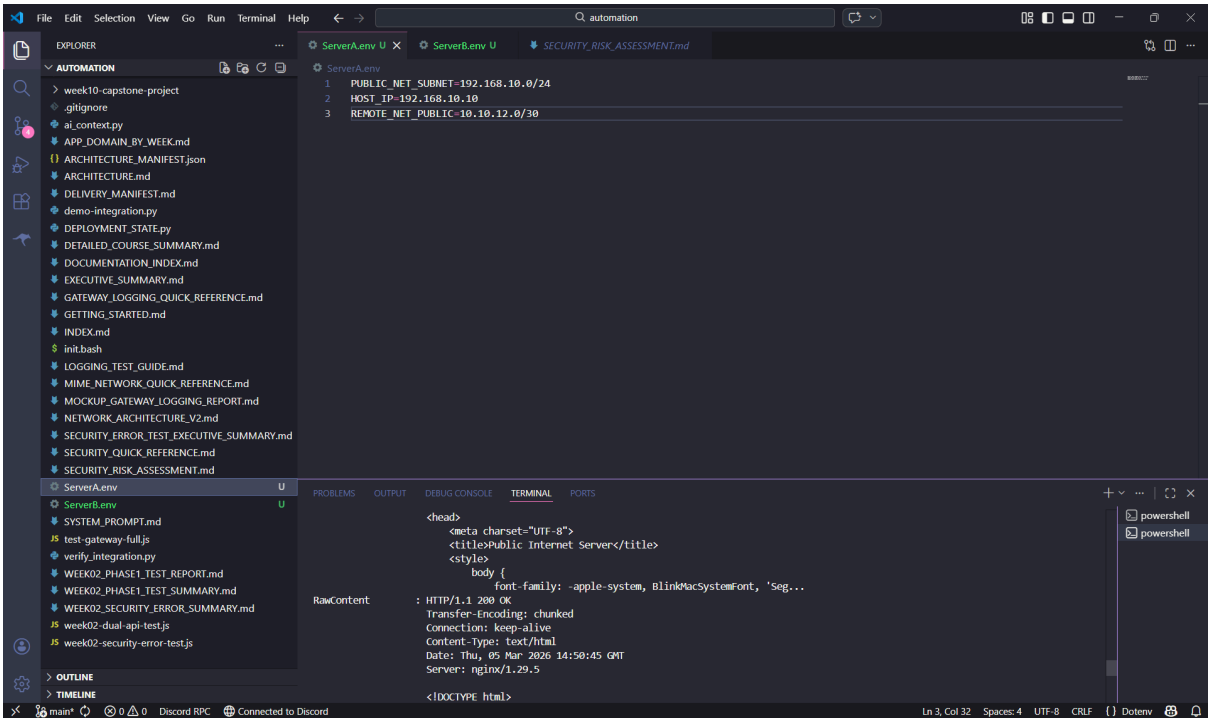
🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

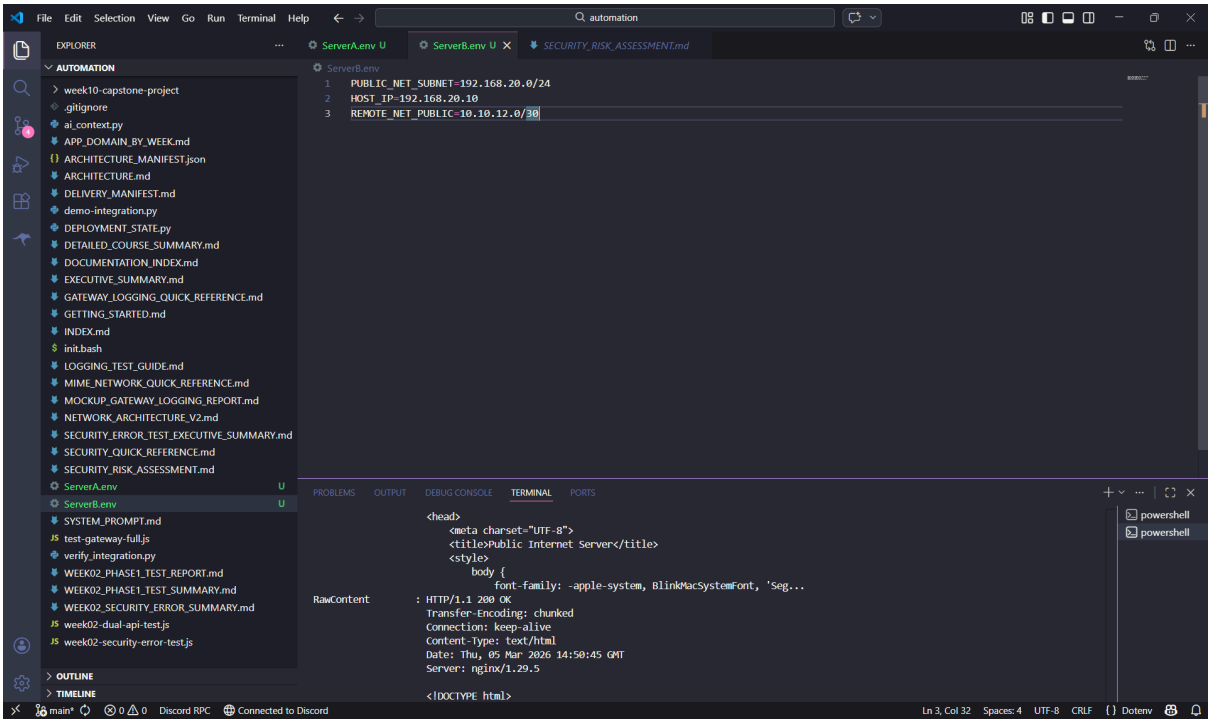
ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```



ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```



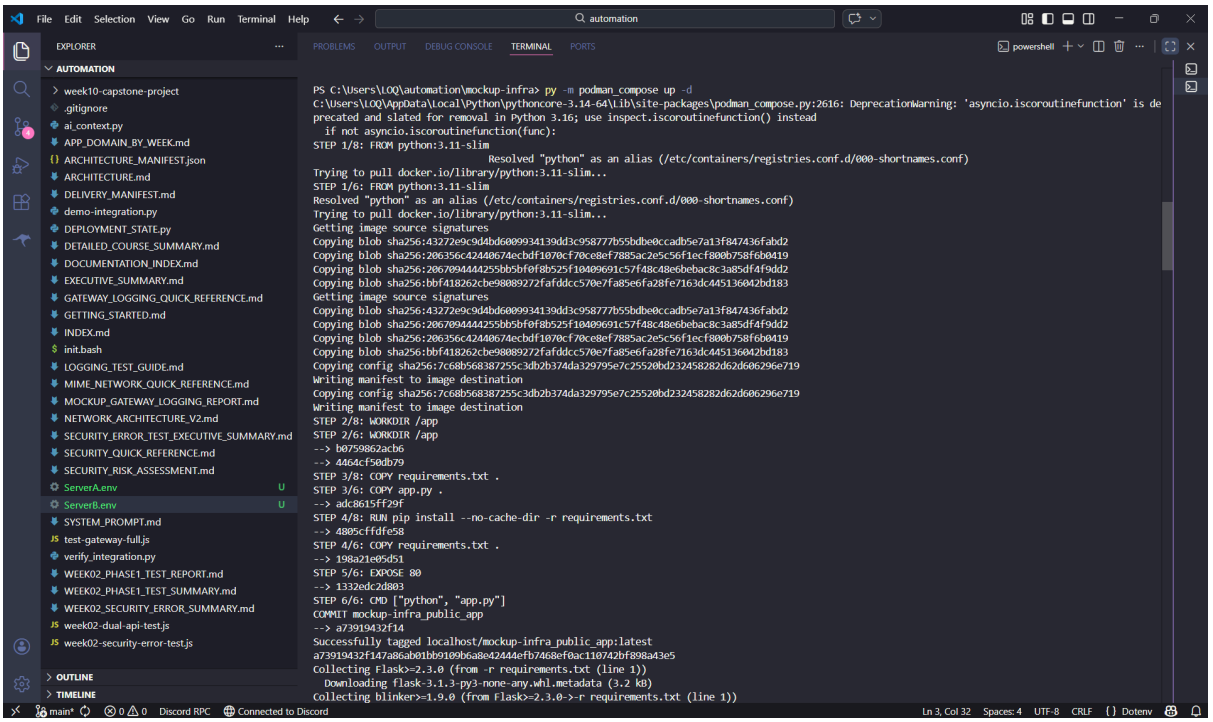
🚀 DEPLOY MOCKUP-INFRA

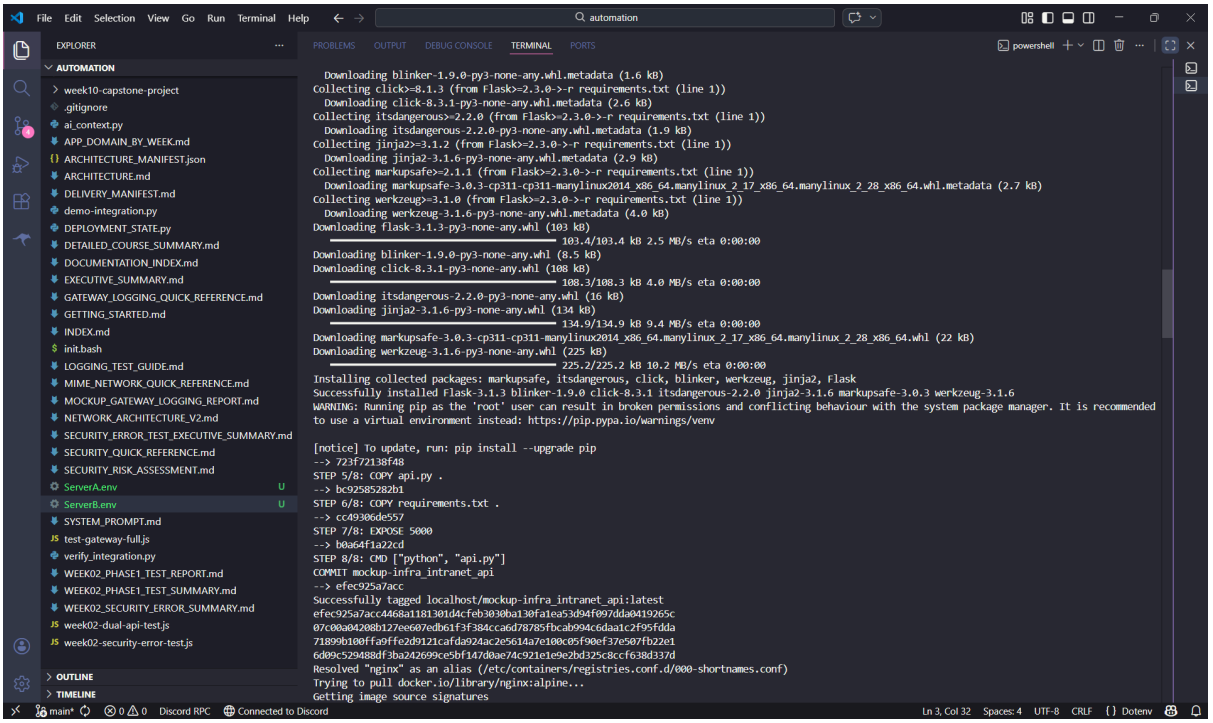
PowerShell (Windows Admin)

None

```
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

```
PS C:\Users\LOQ\automation\mockup-infra> py -m podman_compose up -d
C:\Users\LOQ\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\podman_compose.py:2616: DeprecationWarning: 'asyncio.iscoroutinefunction' is de
precated and slated for removal in Python 3.16; use inspect.iscoroutinefunction() instead
  if not asyncio.iscoroutinefunction(func):
STEP 1/8: FROM python:3.11-slim
                                   Resolved "python" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)
Trying to pull docker.io/library/python:3.11-slim...
STEP 1/6: FROM python:3.11-slim
Resolved "python" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)
Trying to pull docker.io/library/python:3.11-slim...
Getting image source signatures
Copying blob sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2
Copying blob sha256:206356c42440674ecbdf1070cf70ce8ef7885ac2e5c56f1ecf800b758f6b0419
Copying blob sha256:2067094444255bb5bf0f8b525f10409691c57f48c48e6bebac8c3a85df4f9dd2
Copying blob sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183
Getting image source signatures
Copying blob sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2
Copying blob sha256:2067094444255bb5bf0f8b525f10409691c57f48c48e6bebac8c3a85df4f9dd2
Copying blob sha256:206356c42440674ecbdf1070cf70ce8ef7885ac2e5c56f1ecf800b758f6b0419
Copying blob sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183
Copying config sha256:7c68b568387255c3db2b374da329795e7c25520bd232458282d62d606296e719
Writing manifest to image destination
Copying config sha256:7c68b568387255c3db2b374da329795e7c25520bd232458282d62d606296e719
Writing manifest to image destination
STEP 2/8: WORKDIR /app
STEP 2/6: WORKDIR /app
--> b0759862acb6
--> 4464cf50db79
STEP 3/8: COPY requirements.txt .
STEP 3/6: COPY app.py .
--> adc8615ff29f
STEP 4/8: RUN pip install --no-cache-dir -r requirements.txt
--> 4805cffdfe58
STEP 4/6: COPY requirements.txt .
--> 198a21e05d51
STEP 5/6: EXPOSE 80
--> 1332edc2d803
STEP 6/6: CMD ["python", "app.py"]
COMMIT mockup-infra_public_app
--> a73919432f14
Successfully tagged localhost/mockup-infra_public_app:latest
a73919432f147a86ab01bb9109b6a8e42444efb7468ef0ac110742bf898a43e5
Collecting Flask>=2.3.0 (from -r requirements.txt (line 1))
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
```





เนื่องจากเครื่องนี้ใช้คำสั่ง python ตรงๆ ไม่ได้
จึงปรับมาใช้คำสั่ง py -m podman_compose up -d

Bash (Linux)

```shell
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```

🟡 VERIFY CONTAINERS

```none
podman ps
```

```
PS C:\Users\LOQ\automation\mockup-infra> podman ps
CONTAINER ID  IMAGE                                  COMMAND               CREATED        STATUS         PORTS
     NAMES
71899b100ffa  localhost/mockup-infra_public_app:latest  python app.py         22 minutes ago  Up 22 minutes  80/tcp
     mockup-public-web
6d09c529488d  localhost/mockup-infra_intranet_api:latest  python api.py         22 minutes ago  Up 22 minutes  5000/tcp
     mockup-intranet-api
cd185165831d  docker.io/library/nginx:alpine          nginx -g daemon o...  22 minutes ago  Up 22 minutes  0.0.0.0:443->443/tcp, 0.0.0.0:8080->80/
tcp  mockup-gateway
PS C:\Users\LOQ\automation\mockup-infra>
```

Should show:

Container	Port
stateless-api	3000
stateful-api	3001

ผลลัพธ์การทดลอง

- Container สามารถทำงานได้สำเร็จ
- ระบบ Infrastructure ถูกสร้างขึ้นตามที่กำหนด
- Server พร้อมสำหรับการทดสอบ API

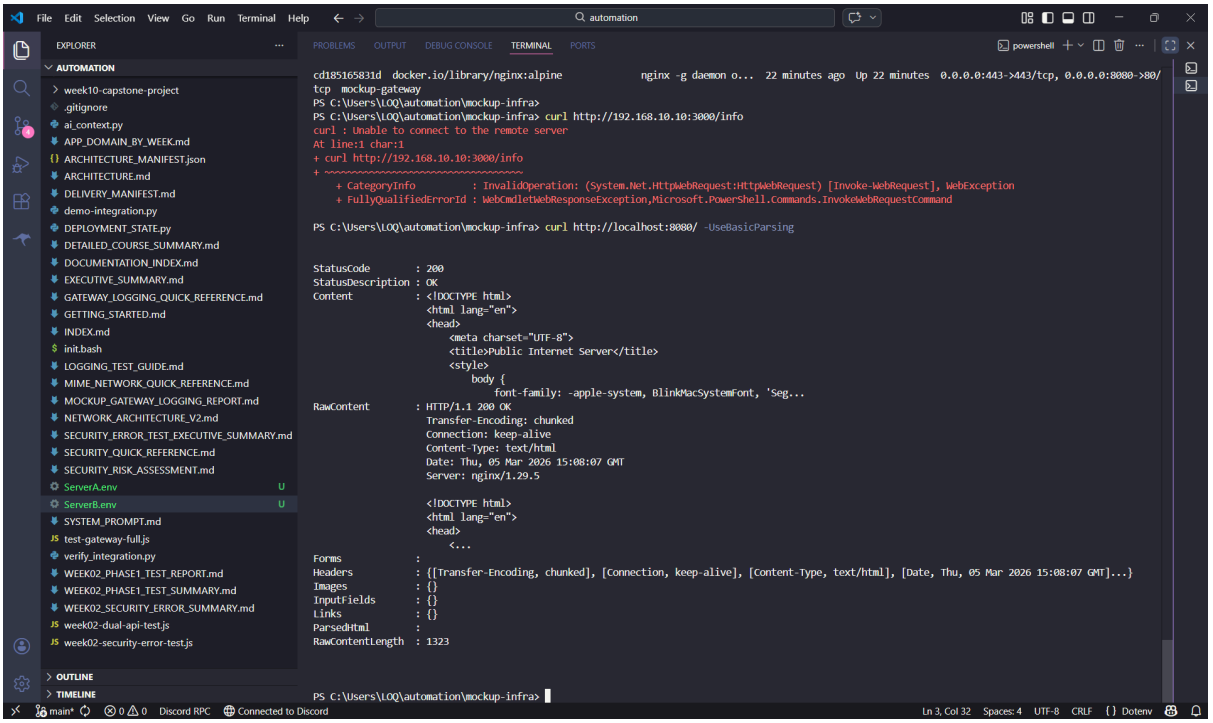
สรุป:
ระบบโครงสร้างพื้นฐานสามารถ Deploy ได้สำเร็จ และมี Container สำหรับ Stateless และ Stateful API ทำงานอยู่

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

⚫ LOCAL TESTS (Private LAN)

From ClientA:

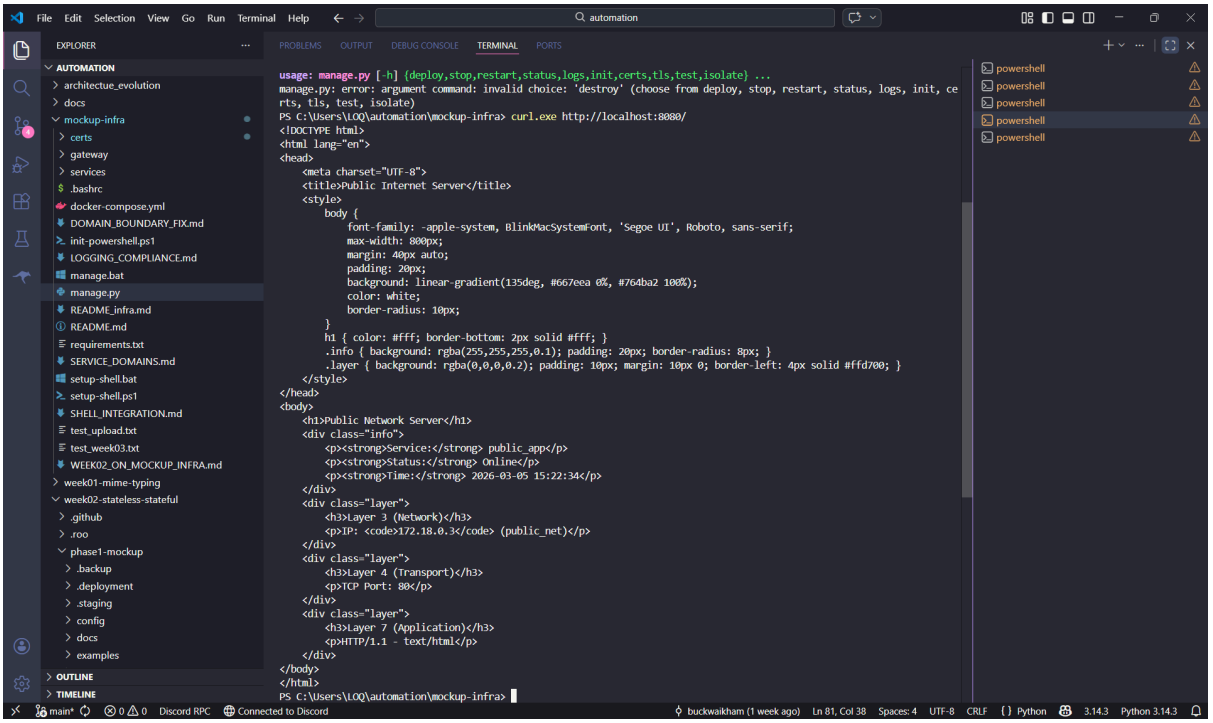
```shell
curl http://192.168.10.10:3000/info
```



→ Stateless

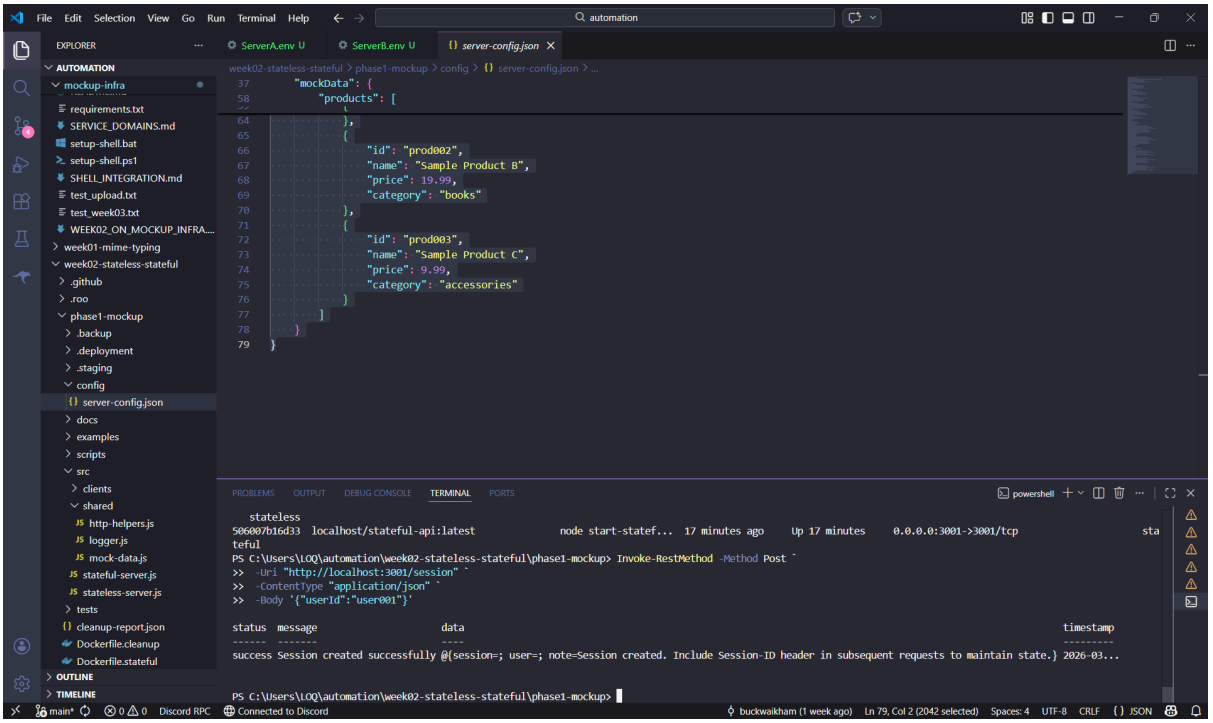
From ClientA:

```shell
curl.exe http://localhost:8080/
```



Store the returned `Session-ID`, then:

```shell
Invoke-RestMethod -Method Post `
 -Uri "http://localhost:3001/session" `
 -ContentType "application/json" `
 -Body '{"userId":"user001"}'
```



🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```
None
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver


```
{
  "server": "ServerB",
  "type": "stateless",
  "status": "running"
}
```

Session Test:

```
None
curl -X POST http://10.10.12.2:3001/session
```


```
{
  "session_id": "abc123xyz"
```

```
}
```
→ ServerB returns a session ID

Then:

```
None
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Check session persists across NAT

```
{
  "session_id": "abc123xyz",
  "status": "active"
}
```

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**

2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```